

Implementacja perceptronu w podejściu obiektowym

Piotr Przymus 127902

1 Cel ćwiczenia

Celem ćwiczenia było zaimplementowanie perceptronu w języku Python z wykorzystaniem podejścia obiektowego oraz sprawdzenie jego działania na przykładzie bramek logicznych AND oraz OR przy użyciu różnych funkcji aktywacji. Dodatkowo przeanalizowano możliwość realizacji bramki XOR.

2 Podstawy teoretyczne

Perceptron jest jednym z najprostszych modeli sztucznej sieci neuronowej. Jego działanie opiera się na obliczeniu sumy ważonej sygnałów wejściowych:

$$z = w_1x_1 + w_2x_2 + b$$

gdzie:

- x_1, x_2 – dane wejściowe,
- w_1, w_2 – wagi,
- b – bias.

Funkcja aktywacji decyduje o końcowym wyniku neuronu. Do funkcji aktywacji należą między innymi:

- funkcja progową (step),
- sigmoid,
- relu.

3 Implementacja

Zaimplementowano klasę `Perceptron`, która umożliwia:

- uczenie modelu (fit),
- predykcję (predict),
- ocenę skuteczności (evaluate).

Funkcja aktywacji została przekazana jako parametr konstruktora klasy.

4 Eksperymenty

W ramach eksperymentów przeanalizowano działanie perceptronu dla trzech bramek logicznych: AND, OR oraz XOR. Dla każdej bramki porównano wyniki uzyskane przy użyciu trzech funkcji aktywacji: step, sigmoid oraz relu.

4.1 Bramka AND

Wejście	Oczekiwany	Predykcja - step	Predykcja - sigmoid	Predykcja - relu
(0,0)	0	0	0	0
(0,1)	0	0	0	0
(1,0)	0	0	0	0
(1,1)	1	1	0	1

Parametry modelu

Funkcja aktywacji	Wagi	Bias
step	[0.2, 0.1]	-0.2
sigmoid	[0.4405, 0.4258]	-1.0589
relu	[0.4716, 0.4458]	-0.1811

Accuracy:

- step: 1.0
- sigmoid: 0.75
- relu: 1.0

Model przy użyciu funkcji aktywacji step oraz relu poprawnie rozwiązuje problem bramki AND. Można zauważyć, że dla funkcji sigmoid model nie osiągnął pełnej skuteczności. Najprawdopodobniej wynika to z zastosowanego sposobu uczenia perceptronu, który został zaprojektowany głównie dla funkcji progowej (step).

4.2 Bramka OR

Wejście	Oczekiwany	Predykcja - step	Predykcja - sigmoid	Predykcja - relu
(0,0)	0	0	1	0
(0,1)	1	1	1	1
(1,0)	1	1	1	1
(1,1)	1	1	1	1

Parametry modelu

Funkcja aktywacji	Wagi	Bias
step	[0.1, 0.1]	-0.1
sigmoid	[0.9786, 0.9888]	0.4068
relu	[0.4005, 0.4258]	0.3387

Accuracy:

- step: 1.0
- sigmoid: 0.75
- relu: 1.0

Podobnie jak dla bramki AND, perceptron z funkcjami aktywacji step i relu poprawnie rozwiązuje problem bramki OR, a funkcja sigmoid radzi sobie gorzej.

4.3 Bramka XOR

Wejście	Oczekiwany	Predykcja - step	Predykcja - sigmoid	Predykcja - relu
(0,0)	0	1	0	0
(0,1)	1	1	0	0
(1,0)	1	0	0	0
(1,1)	0	0	0	0

Parametry modelu

Funkcja aktywacji	Wagi	Bias
step	[-0.1, 0.0]	0.0
sigmoid	[-0.0414, -0.0208]	-0.0032
relu	[-0.0530, -0.0007]	0.4792

Accuracy:

- step: 0.5
- sigmoid: 0.5
- relu: 0.5

Niezależnie od użytej funkcji aktywacji perceptron nie był w stanie poprawnie nauczyć się pełnej zależności wejście-wyjście dla bramki XOR. Wynika to z faktu, że problem XOR nie jest liniowo separowalny, co jest fundamentalnym ograniczeniem perceptronu jednowarstwowego.

5 Analiza problemu XOR

Model działa bardzo dobrze dla problemów liniowo separowalnych, takich jak bramki AND oraz OR. Oznacza to, że da się znaleźć jedną prostą (w praktyce: linię na wykresie), która oddziela przypadki należące do klasy 0 i 1.

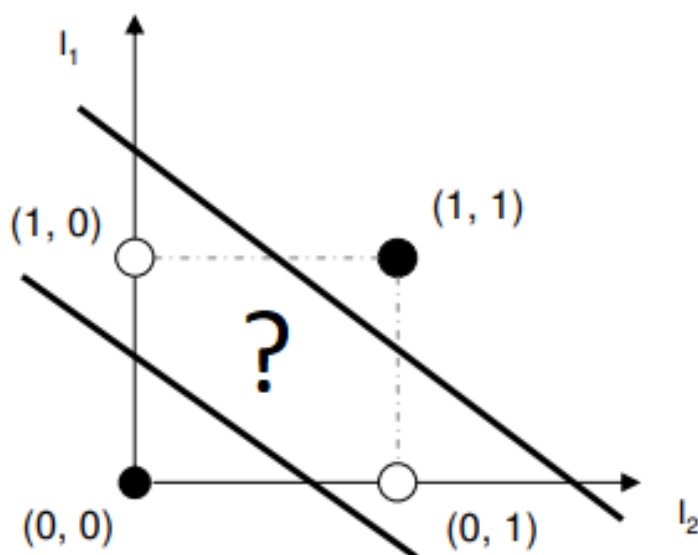
W przypadku bramki XOR sytuacja jest inna. Dane wejściowe XOR mają taką strukturę, że nie da się ich rozdzielić jedną prostą linią. Jeśli narysujemy punkty (0,0), (0,1), (1,0), (1,1), to klasy 0 i 1 są „przeplatane” po przekątnej. Oznacza to, że każda pojedyncza linia podziału zawsze pomyli przynajmniej jeden punkt.

Perceptron jednowarstwowy działa dokładnie w taki sposób, że podejmuje decyzję na podstawie jednej takiej linii (hiperpłaszczyzny) opisanej równaniem:

$$w_1x_1 + w_2x_2 + b = 0$$

Dlatego niezależnie od tego, jak dobierzemy wagi, bias albo funkcję aktywacji, model nadal próbuje oddzielić dane jedną prostą granicą decyzyjną. To powoduje, że nie jest w stanie poprawnie nauczyć się XOR.

W praktyce oznacza to, że problem nie leży w implementacji ani w funkcji aktywacji, tylko w samej architekturze modelu - jest ona zbyt prosta. Jak można zaobserwować na rysunku 1, klasy XOR nie mogą zostać rozdzielone jedną linią prostą. Aby uzyskać poprawną separację, konieczne jest zastosowanie co najmniej dwóch linii decyzyjnych, co wykracza poza możliwości pojedynczego perceptronu. Z tego powodu do rozwiązania XOR potrzebna jest sieć neuronowa z co najmniej jedną warstwą ukrytą, która potrafi tworzyć bardziej złożone, nieliniowe granice decyzyjne.



Rysunek 1: Wizualizacja problemu XOR. Punkty (0,0) i (1,1) należą do klasy 0, a punkty (0,1) i (1,0) należą do klasy 1. Nie można oddzielić tych klas jedną prostą linią.

6 Wnioski

1. Perceptron został poprawnie zaimplementowany i działa zgodnie z założeniami teoretycznymi. Potrafi uczyć się na podstawowych danych wejściowych i dostosowywać wagi oraz bias.
2. Model bardzo dobrze radzi sobie z prostymi problemami, które można rozdzielić jedną linią, takimi jak bramki AND oraz OR. W takich przypadkach osiąga wysoką skuteczność niezależnie od funkcji aktywacji.
3. Funkcje aktywacji step, sigmoid oraz relu wpływają na sposób działania modelu, jednak nie zmieniają jego podstawowych ograniczeń. W niektórych przypadkach sigmoid może pogarszać wyniki w zadaniach binarnych.

4. Problem XOR nie został poprawnie rozwiązany przez perceptron, ponieważ dane nie są liniowo separowalne. Oznacza to, że nie da się ich poprawnie rozdzielić jedną prostą granicą decyzyjną.
5. Najważniejszym ograniczeniem perceptronu nie jest funkcja aktywacji, ale jego liniowa struktura, która pozwala na tworzenie tylko prostych granic decyzyjnych.
6. Aby rozwiązać bardziej złożone problemy, takie jak XOR, konieczne jest zastosowanie bardziej zaawansowanych modeli, np. sieci neuronowych z warstwami ukrytymi.